

TABLE OF CONTENTS

1	Aim	3
2	Components Required	3
3	Role of Components	3
3.1	Arduino	3
3.2	ZMPT101B Voltage Sensor	3
3.3	SCT-013 Current Transformer	3
3.4	Relay Module	4
3.5	Capacitor Bank	4
4	Microcontroller: Arduino (ATmega328P)	4
4.1	Why Arduino?	4
4.2	Key Specifications	4
5	Block Diagrams	5
5.1	System Architecture	5
5.2	System Flow	6
6	Code Listings	7
6.1	Main Arduino Program	7
7	Key Idea	9
8	System Description	9
9	Working	9
10	Mathematical Formulation	10
11	Applications	11
12	Conclusion	11

1 Aim

The aim of this project is to design and implement a low-cost Automatic Power Factor Correction (APFC) system that follows industrial measurement principles by accurately sensing voltage and current, computing true power factor using DSP-style sampling, and dynamically switching capacitor banks through relays to improve the power factor of inductive loads in real time.

2 Components Required

- Arduino Uno (ATmega328P)
- ZMPT101B voltage sensor
- SCT-013 current transformer
- Relay module
- Capacitor bank
- Inductive test load (fan / motor / RL load for demonstration)
- Power supply and wiring

3 Role of Components

3.1 Arduino

The Arduino acts as the central controller of the entire APFC system. It acquires sampled voltage and current signals from the sensors through its ADC channels, performs offset removal and calibration, computes RMS values and instantaneous power, determines the true power factor, estimates required reactive power compensation, and drives the relay outputs that switch the capacitor bank.

3.2 ZMPT101B Voltage Sensor

The ZMPT101B module is used to safely sense the AC mains voltage waveform. It provides an isolated low-voltage analog representation of the AC voltage that can be sampled by the Arduino ADC for waveform analysis and RMS voltage computation.

3.3 SCT-013 Current Transformer

The SCT-013 current transformer senses the load current in a non-intrusive and isolated manner. It produces a proportional current/voltage signal that can be conditioned and sampled by the Arduino for RMS current and power computation.

3.4 Relay Module

The relay module provides electrical isolation between the low-voltage controller and the capacitor bank switching path. It allows the Arduino to connect or disconnect individual capacitor steps depending on the required reactive power compensation.

3.5 Capacitor Bank

The capacitor bank supplies leading reactive power to compensate for the lagging reactive power drawn by inductive loads. By switching capacitor steps in and out, the system improves the power factor and reduces unnecessary current draw from the supply.

4 Microcontroller: Arduino (ATmega328P)

4.1 Why Arduino?

- Easy analog interfacing with multiple ADC channels for voltage and current sensing
- Simple and rapid prototyping suitable for microcontroller laboratory projects
- Sufficient processing capability for moderate-rate DSP-style sampling and calculations
- Large ecosystem of libraries and debugging support
- Low cost and easy availability compared to industrial power analyzer hardware
- Adequate GPIO pins for relay control and optional LCD / serial monitoring

4.2 Key Specifications

Parameter	Specification
Microcontroller	ATmega328P
Clock Speed	16 MHz
Operating Voltage	5V
Analog Inputs	6 channels (10-bit ADC)
Digital I/O	14 pins
Flash Memory	32 KB
SRAM	2 KB

5 Block Diagrams

5.1 System Architecture

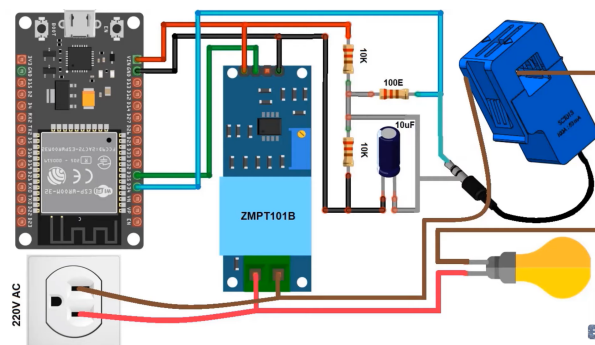
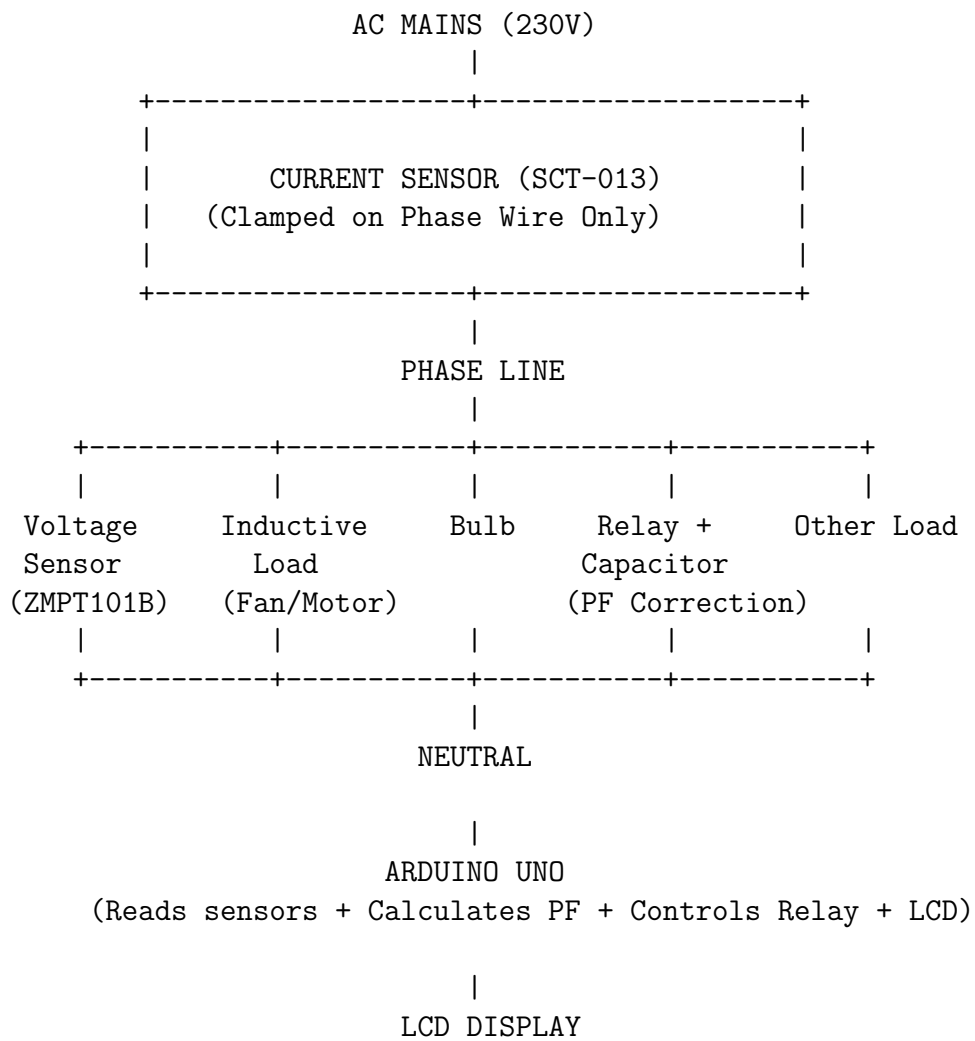
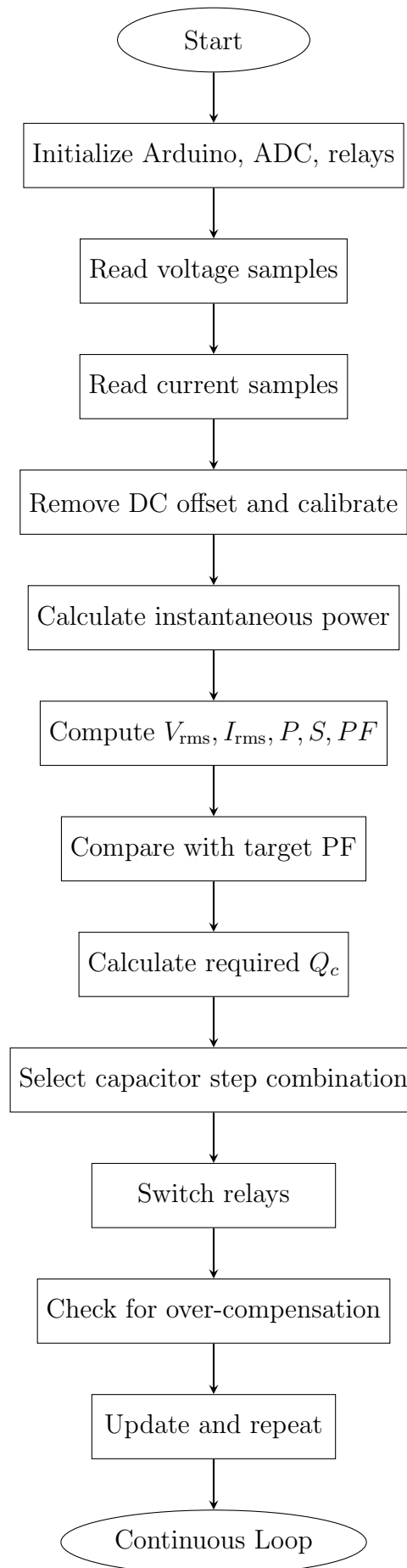


Figure 1: Circuit diagram

5.2 System Flow



6 Code Listings

6.1 Main Arduino Program

```
1  #include <LiquidCrystal.h>
2  #include "EmonLib.h"
3
4  int relay1 = 9;
5  int relay2 = 10;
6
7  int input1 = 11;
8  int input2 = 12;
9
10 LiquidCrystal lcd(7, 6, 5, 4, 3, 2);
11 EnergyMonitor emon;
12
13 #define vCalibration 280.0
14 #define currCalibration 16.5
15 #define phaseCalibration 0.05
16
17 float kWh = 0;
18 unsigned long lastmillis;
19
20 void setup()
21 {
22     pinMode(relay1, OUTPUT);
23     pinMode(relay2, OUTPUT);
24
25     pinMode(input1, INPUT_PULLUP);
26     pinMode(input2, INPUT_PULLUP);
27
28     Serial.begin(9600);
29     lcd.begin(16, 2);
30
31     // Initialize sensors
32     emon.voltage(A0, vCalibration, phaseCalibration);
33     emon.current(A1, currCalibration);
34
35     lcd.setCursor(0, 0);
36     lcd.print("Energy_Meter");
37     delay(2000);
38
39     lastmillis = millis();
40     digitalWrite(relay1, HIGH); // OFF
41     digitalWrite(relay2, HIGH); // OFF
42 }
43
44 void loop()
45 {
46     emon.calcVI(40, 2000);
47
```

```

48 unsigned long currentMillis = millis();
49 float elapsedHours = (currentMillis - lastmillis) / 3600000.0;
50
51 kWh += emon.realPower * elapsedHours / 1000.0;
52 lastmillis = currentMillis;
53
54 Serial.print("Vrms:");
55 Serial.print(emon.Vrms, 2);
56 Serial.print("\t");
57
58 Serial.print("Irms:");
59 Serial.print(emon.Irms, 3);
60 Serial.print("\t");
61
62 Serial.print("Power:");
63 Serial.print(emon.realPower, 2);
64 Serial.print("\t");
65
66 Serial.print("PF:");
67 Serial.print(emon.powerFactor, 3);
68
69 Serial.print("\tkWh:");
70 Serial.println(kWh, 4);
71
72 lcd.setCursor(0, 0);
73 lcd.print("V:");
74 lcd.print(emon.Vrms, 1);
75 lcd.print(" ");
76
77 lcd.setCursor(9, 0);
78 lcd.print("I:");
79 lcd.print(emon.Irms, 2);
80 lcd.print(" ");
81
82 lcd.setCursor(0, 1);
83 lcd.print("P:");
84 lcd.print(emon.realPower, 1);
85 lcd.print(" ");
86
87 lcd.setCursor(9, 1);
88 lcd.print("PF:");
89 lcd.print(emon.powerFactor, 3);
90 lcd.print(" ");
91
92 delay(500);
93
94 if (digitalRead(input1) == LOW) {
95     digitalWrite(relay1, LOW);    // ON
96 } else {
97     digitalWrite(relay1, HIGH);  // OFF
98 }

```

```

99
100  if (digitalRead(input2) == LOW) {
101      digitalWrite(relay2, LOW);    // ON
102  } else {
103      digitalWrite(relay2, HIGH);   // OFF
104  }
105  }

```

7 Key Idea

The key idea of this project is to implement a low-cost APFC system that behaves closer to an industrial power monitoring and correction unit rather than a simple academic phase-angle demonstration. Instead of estimating only displacement power factor using zero-crossing methods, the system samples both voltage and current waveforms, calculates instantaneous power, and derives true power factor using the relationship between real and apparent power. This makes the design more robust in the presence of waveform distortion, harmonics, and non-ideal practical conditions.

8 System Description

The system is organized into four major functional blocks: sensing, signal processing, decision making, and correction. In the sensing stage, the ZMPT101B measures the AC voltage waveform while the SCT-013 measures the load current. In the signal processing stage, the Arduino samples both waveforms, removes offsets, applies calibration factors, and computes electrical parameters such as V_{rms} , I_{rms} , real power, apparent power, reactive power, and power factor.

In the decision-making stage, the controller compares the measured power factor with a predefined target value such as 0.95 lagging. Based on the difference, it estimates the reactive power that must be supplied by the capacitor bank. In the correction stage, the relay module switches one or more capacitor steps to inject leading reactive power and improve the overall system power factor.

9 Working

The system starts by initializing the Arduino, sensor interfaces, and relay outputs. Once the inductive load is connected, the voltage and current sensors begin providing analog signals proportional to the mains voltage and load current.

The Arduino continuously samples both signals over multiple AC cycles. Each pair of voltage and current samples is used to calculate instantaneous power. After collecting enough samples, the firmware calculates average real power, RMS voltage, RMS current, and apparent power. The true power factor is then determined using $PF = P/S$.

If the measured power factor is lower than the desired value, the controller calculates the reactive power of the load and then determines the capacitor reactive power required for correction. Based on this value, a suitable combination of capacitor steps is selected. The relay module energizes the corresponding channels and connects the capacitor bank to the circuit. This reduces lagging reactive power, improves the power factor, and lowers

the line current. The process repeats continuously so that the system adapts to changes in the load.

10 Mathematical Formulation

The mathematical formulation used in this project is summarized below:

Instantaneous Power

$$p(t) = v(t) i(t) \quad (1)$$

Average Real Power

$$P = \frac{1}{N} \sum_{k=1}^N v_k i_k \quad (2)$$

RMS Voltage

$$V_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{k=1}^N v_k^2} \quad (3)$$

RMS Current

$$I_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{k=1}^N i_k^2} \quad (4)$$

Apparent Power

$$S = V_{\text{rms}} \cdot I_{\text{rms}} \quad (5)$$

True Power Factor

$$PF = \frac{P}{S} \quad (6)$$

Reactive Power

$$Q = P \tan(\cos^{-1}(PF)) \quad (7)$$

Required Capacitive Compensation

$$Q_c = P (\tan \phi_1 - \tan \phi_2) \quad (8)$$

11 Applications

- Industrial power systems
- Energy monitoring
- Educational laboratories
- Smart distribution panels

12 Conclusion

This project successfully demonstrates a low-cost Automatic Power Factor Correction system based on industrial measurement principles. By using voltage and current waveform sampling instead of simple phase-angle estimation, the system computes true power factor more accurately and performs intelligent capacitor switching through a relay-controlled capacitor bank. The design is economical, practical, and educationally valuable, making it suitable for microcontroller laboratory demonstration, prototype APFC development, and small-scale embedded energy management applications.